

MISIÓN

Construye desde cero un sistema interno de control de stock para una empresa pequeña.

La aplicación se ejecutará LOCALMENTE dentro de la red LAN de la empresa en una PC que permanece encendida. No necesitamos acceso público desde Internet.

MongoDB Atlas será la base de datos remota y la fuente de verdad del inventario confirmado.

La aplicación debe seguir funcionando cuando se pierda temporalmente la conexión a Internet/MongoDB Atlas, sin perder movimientos ni duplicarlos cuando vuelva la conectividad.

El sistema será utilizado principalmente para controlar mercadería retirada por técnicos.

La prioridad absoluta es:

1. integridad de datos
2. seguridad
3. trazabilidad
4. prevención de duplicados
5. tolerancia a fallos
6. funcionamiento offline
7. UX
8. mantenibilidad
9. performance
10. simplicidad arquitectónica

No quiero un prototipo descartable.

Quiero una aplicación funcional, mantenible, segura y lista para uso real.

1. MODO DE TRABAJO — AUTOPILOT

Estás trabajando con permisos Autopilot.

Trabaja de manera autónoma.

Puedes:

- crear archivos
- modificar archivos
- mover archivos
- eliminar archivos de código obsoletos cuando sea necesario

- instalar dependencias estrictamente necesarias
- ejecutar comandos
- ejecutar tests
- ejecutar lint
- ejecutar builds
- utilizar Docker
- utilizar Docker Compose
- refactorizar código cuando mejore objetivamente el proyecto

No pidas confirmaciones para decisiones técnicas normales que puedan resolverse razonablemente a partir de esta especificación.

Si falta un detalle menor:

- toma una decisión razonable
- prioriza simplicidad
- documenta decisiones importantes
- continúa

NO:

- cambies el stack definido
- introduzcas microservicios
- introduzcas NATS
- introduzcas Redis
- introduzcas Kafka
- introduzcas RabbitMQ
- introduzcas servicios pagos
- elimines requisitos para simplificar
- escribas secrets reales
- expongas credenciales
- ejecute operaciones destructivas fuera del repositorio
- alteres bases de datos de producción
- uses `git reset --hard`
- uses `git clean -fd`
- uses `git push --force`
- alteres historial Git

salvo instrucción explícita del usuario.

No declares una funcionalidad terminada solamente porque compila.

Debe:

- funcionar
- estar probada
- manejar errores
- respetar seguridad

- respetar invariantes
 - no romper funcionalidades existentes
-

2. STACK OBLIGATORIO

Frontend

- React
- Vite
- JavaScript
- Tailwind CSS
- React Router
- IndexedDB cuando sea útil para resiliencia del navegador

Backend

- Node.js
- Express
- JavaScript
- Mongoose
- Zod
- MongoDB Atlas

Persistencia local

Utilizar SQLite en el backend exclusivamente para:

- durable outbox
- operaciones pendientes
- estado de sincronización
- metadata necesaria para recuperación

SQLite NO reemplaza MongoDB.

MongoDB Atlas sigue siendo la fuente de verdad del inventario confirmado.

Infraestructura

- Docker
- Docker Compose

La aplicación correrá en una PC local dentro de la empresa.

Testing

- Vitest
- React Testing Library
- Supertest
- Playwright

Calidad

- ESLint
- Prettier
- GitHub Actions

No utilizar TypeScript.

No utilizar Next.js.

No utilizar Redux salvo que exista una necesidad real demostrable.

No utilizar Bootstrap.

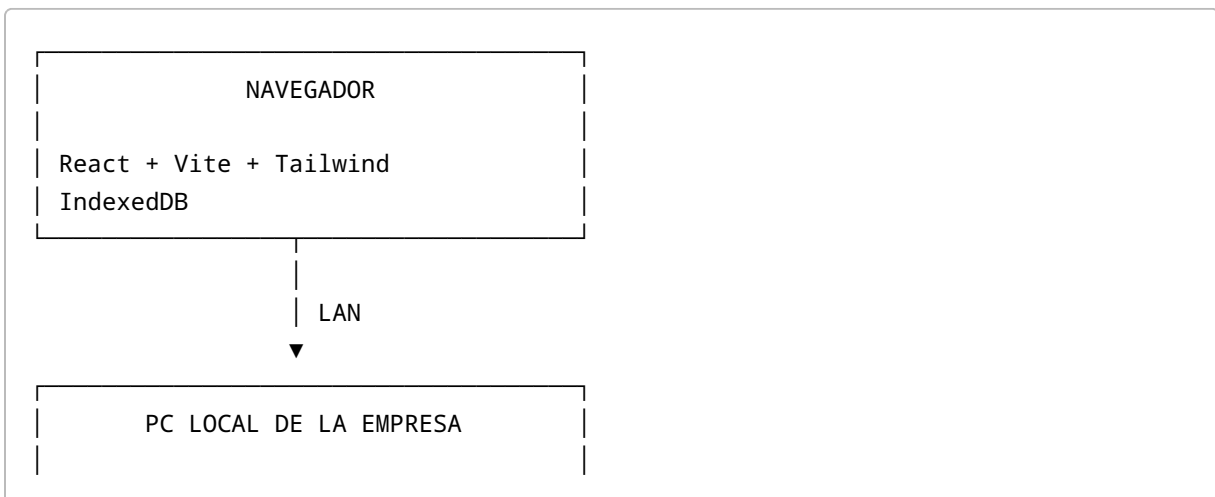
No utilizar Material UI.

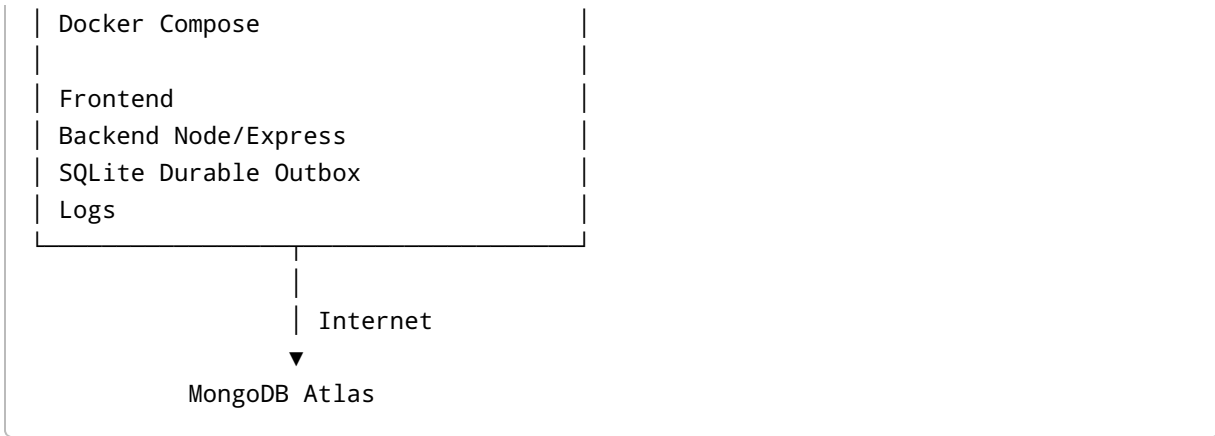
No utilizar microservicios.

La arquitectura será un MONOLITO MODULAR.

3. ARQUITECTURA GENERAL

Conceptualmente:





4. ESTRUCTURA DEL PROYECTO

Utilizar aproximadamente:

```
stock-control/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   │   ├── ui/  
│   │   │   ├── scanner/  
│   │   │   ├── inventory/  
│   │   │   ├── products/  
│   │   │   └── layout/  
│   │   ├── pages/  
│   │   ├── features/  
│   │   │   ├── auth/  
│   │   │   ├── products/  
│   │   │   ├── inventory/  
│   │   │   ├── movements/  
│   │   │   ├── users/  
│   │   │   ├── audit/  
│   │   │   └── sync/  
│   │   ├── hooks/  
│   │   ├── services/  
│   │   ├── offline/  
│   │   │   ├── indexedDb.js  
│   │   │   └── connectivity.js  
│   │   ├── router/  
│   │   └── utils/  
└──
```

```
|
|
|   |─ App.jsx
|   |─ main.jsx
|
|   └─ tests/
|
|─ backend/
|   |─ src/
|   |   |─ config/
|   |   |─ modules/
|   |   |   |─ auth/
|   |   |   |─ users/
|   |   |   |─ products/
|   |   |   |─ inventory/
|   |   |   |─ locations/
|   |   |   |─ categories/
|   |   |   |─ audit/
|   |   |   └─ sync/
|   |
|   |   |─ middlewares/
|   |   |─ shared/
|   |   |   |─ errors/
|   |   |   |─ logger/
|   |   |   |─ validation/
|   |   |   |─ security/
|   |   |   └─ utils/
|   |
|   |   |─ infrastructure/
|   |   |   |─ mongodb/
|   |   |   └─ sqlite/
|   |
|   |   |─ app.js
|   |   └─ server.js
|   |
|   └─ tests/
|
|─ shared/
|   └─ schemas/
|
|─ scripts/
|─ docker/
|─ .github/
|   └─ workflows/
|─ docker-compose.yml
|─ .env.example
|─ .gitignore
```

```
├─ README.md
└─ package.json
```

No sigas esta estructura ciegamente si alguna pequeña modificación mejora claramente la organización.

Mantén, sin embargo, separación clara por módulos.

5. ESTRUCTURA DE MÓDULOS BACKEND

Ejemplo:

```
inventory/
├─ inventory.model.js
├─ inventory.repository.js
├─ inventory.service.js
├─ inventory.controller.js
├─ inventory.routes.js
├─ inventory.validation.js
└─ inventory.errors.js
```

Responsabilidades:

Routes

Solo:

- endpoints
- middlewares
- routing

Controllers

Solo:

- adaptar HTTP
- obtener parámetros
- llamar services
- devolver respuestas

Services

Contienen:

- lógica de negocio
- reglas
- invariantes
- coordinación de operaciones

Repositories

Contienen:

- acceso a persistencia
- Mongoose
- queries

Models

Contienen:

- schemas Mongoose
- índices
- restricciones

Validation

Zod.

Nunca colocar lógica de negocio significativa en:

- routes
- controllers
- componentes React
- middlewares

6. OBJETIVO FUNCIONAL

Controlar:

- productos
- stock
- entradas
- salidas

- devoluciones
- ajustes
- usuarios
- ubicaciones
- auditoría
- sincronización
- conteos físicos

La operación más importante es:

REGISTRAR SALIDA.

Cada técnico tendrá su propio usuario.

Debe iniciar sesión antes de retirar mercadería.

Cada salida debe permitir determinar inequívocamente:

- quién retiró
- qué retiró
- cuánto retiró
- cuándo
- a qué hora
- por qué
- para qué trabajo/cliente si corresponde
- stock anterior
- stock resultante
- estado de sincronización
- operationId

7. ROLES

Inicialmente:

ADMIN

Puede:

- consultar productos
- crear productos
- actualizar productos
- desactivar productos
- reactivar productos
- registrar entradas
- registrar salidas

- registrar devoluciones
- realizar ajustes
- realizar conteos físicos
- consultar todos los movimientos
- administrar usuarios
- administrar categorías
- administrar ubicaciones
- consultar auditoría
- consultar estado del sistema
- resolver conflictos mediante operaciones explícitas

TECHNICIAN

Puede:

- iniciar sesión
- consultar productos
- consultar stock disponible
- registrar salidas
- registrar devoluciones permitidas
- consultar sus propios movimientos

No puede:

- registrar entradas
- ajustar stock
- administrar usuarios
- modificar productos
- consultar auditoría completa

Los permisos deben validarse SIEMPRE en backend.

Ocultar un botón en React NO constituye seguridad.

8. NO EXISTEN BORRADOS FÍSICOS

Esta es una INVARIANTE del sistema.

No implementar DELETE físico para información funcional.

Movimientos

Son APPEND-ONLY.

Nunca:

- borrar
- editar
- sobrescribir

Productos

Operaciones:

- crear
- consultar
- actualizar
- desactivar
- reactivar

No implementar DELETE.

Utilizar:

```
active: false
```

Usuarios

No borrar.

Desactivar.

Categorías

No borrar si están o estuvieron referenciadas.

Archivar/desactivar.

Ubicaciones

Igual.

Audit logs

Inmutables.

No debe existir:

```
DELETE /movements/:id
```

ni equivalente.

9. CORRECCIONES

Los errores históricos nunca se corrigen modificando el movimiento original.

Tipos mínimos:

```
IN  
OUT  
RETURN  
ADJUSTMENT_IN  
ADJUSTMENT_OUT
```

Ejemplo:

Se registró:

```
OUT 5
```

pero correspondían 4.

NO:

```
editar OUT 5 → OUT 4
```

Crear:

```
RETURN 1
```

o:

```
ADJUSTMENT_IN 1
```

según corresponda.

Debe quedar:

- movimiento original
- movimiento correctivo
- usuario que corrigió
- fecha
- hora
- motivo
- relación entre movimientos

10. MODELO PRODUCT

Como mínimo:

```
{
  _id,
  internalCode,
  barcodes: [],
  name,
  brand,
  model,
  categoryId,
  locationId,
  stock,
  minimumStock,
  serializable,
  active,
  createdAt,
  updatedAt
}
```

Permitir múltiples códigos de barras cuando corresponda.

`internalCode` debe ser único.

Ejemplos:

```
CAM-000001
DVR-000001
NVR-000001
SEN-000001
ACC-000001
```

Un producto sin barcode comercial debe seguir siendo perfectamente utilizable.

Dejar preparada la posibilidad de generar posteriormente etiquetas Code 128.

11. PRODUCTOS SERIALIZABLES

Preparar arquitectura para productos con número de serie.

Ejemplos:

- cámaras
- DVR
- NVR
- centrales
- módulos

No todos los productos requieren serial.

El diseño debe permitir incorporar:

```
serialNumber
```

sin rehacer la arquitectura.

12. UBICACIONES

Modelar ubicación como entidad.

Aunque actualmente exista un único depósito.

Permitir conceptos como:

```
depósito  
estantería  
nivel  
posición
```

Ejemplo:

13. SALIDAS

Motivo obligatorio.

Motivos iniciales:

Instalación
Mantenimiento
Service
Préstamo
Uso interno
Otro

Permitir opcionalmente:

- cliente
- número de trabajo
- observación

No hacer obligatorios campos que no siempre sean necesarios.

14. STOCK

Nunca utilizar:

```
const product = await Product.findById(id);  
  
product.stock -= quantity;  
  
await product.save();
```

Esto genera race conditions.

Usar operaciones atómicas.

Para OUT:

```
{
  _id: productId,
  stock: { $gte: quantity }
}
```

y:

```
{
  $inc: {
    stock: -quantity
  }
}
```

Nunca permitir:

```
stock < 0
```

Validar:

```
quantity > 0
```

15. TRANSACCIONES

Las operaciones críticas deben ser transaccionales cuando MongoDB esté disponible.

Una salida confirmada debe coordinar:

```
validar idempotencia
↓
validar usuario
↓
validar producto
↓
validar permisos
↓
validar cantidad
↓
validar stock
↓
```



```
actualizar stock
↓
crear movimiento
↓
crear auditoría
↓
COMMIT
```

Si algo falla:

```
ROLLBACK
```

Nunca dejar:

```
stock modificado
movimiento ausente
```

o viceversa.

16. IDEMPOTENCIA

Cada operación debe tener:

```
operationId
```

Generado con:

```
crypto.randomUUID()
```

Debe existir índice UNIQUE en MongoDB.

Ejemplo:

```
stockMovementSchema.index(
  { operationId: 1 },
  { unique: true }
);
```

Una misma operationId nunca puede afectar stock dos veces.

Debe funcionar correctamente ante:

- doble click
- doble escaneo
- retry HTTP
- timeout
- pérdida de respuesta
- reconexión
- reenvío desde outbox
- reinicio
- múltiples intentos

Si MongoDB procesó:

```
operationId ABC
```

pero la respuesta nunca llegó, un retry de ABC debe reconocer que ya fue procesada.

No volver a descontar stock.

17. DUPLICADOS

Usar índices UNIQUE reales.

No confiar solamente en:

```
if (!existing)
```

Proteger:

- operationId
- internalCode
- barcodes
- email

Manejar correctamente errores de duplicate key.

18. OFFLINE-FIRST

Este requisito es CRÍTICO.

Offline en este proyecto significa principalmente:

LAN	OK
PC servidor	OK
Node	OK
MongoDB Atlas	NO DISPONIBLE

El sistema debe permitir registrar operaciones sin perderlas.

19. DURABLE OUTBOX

Utilizar SQLite local en backend como durable outbox.

La primera garantía de persistencia durante una caída de Atlas debe ser local.

Flujo:

```
React
↓
Node local
↓
persistir operación durablemente
↓
intentar MongoDB
```

Si MongoDB responde:

```
procesar
↓
SYNCED
```

Si MongoDB no responde:

```
operación permanece en SQLite
↓
PENDING
```

Debe sobrevivir:

- reinicio Node

- reinicio Docker
- reinicio PC
- cierre del navegador

Nunca depender exclusivamente de RAM.

20. MONGODB ES LA FUENTE DE VERDAD

SQLite NO es una segunda base de inventario.

Nunca hacer:

```
stock SQLite = X
↓
sobrescribir MongoDB con X
```

La outbox almacena INTENCIONES/OPERACIONES.

Ejemplo:

```
Usuario X quiere retirar 3 unidades de producto Y
```

No:

```
El nuevo stock definitivo es 17
```

Cuando vuelva MongoDB:

```
leer operación pendiente
↓
evaluarla contra stock ACTUAL
↓
aplicar si sigue siendo válida
↓
o generar CONFLICT
```

21. ESTADOS DE SINCRONIZACIÓN

Como mínimo:

PENDING
SYNCING
SYNCED
FAILED
CONFLICT

Definir claramente las transiciones válidas.

No permitir estados imposibles.

22. SYNC MANAGER

Implementar worker/manager de sincronización.

Debe:

- detectar disponibilidad MongoDB
- obtener pendientes
- bloquear/claim operaciones de forma segura
- procesarlas
- usar idempotencia
- utilizar retry
- aplicar exponential backoff razonable
- evitar loops agresivos
- distinguir errores transitorios
- distinguir conflictos
- registrar intentos
- recuperar operaciones `SYNCING` abandonadas después de crash
- sobrevivir reinicios

No deben existir dos workers procesando accidentalmente la misma operación sin protección.

23. CONFLICTOS OFFLINE

Ejemplo:

MongoDB:

```
stock = 3
```

Offline:

```
Técnico A → OUT 2  
Técnico B → OUT 2
```

Al sincronizar:

```
A  
3 → 1  
SYNCED
```

B:

```
disponible 1  
solicitado 2
```

Resultado:

```
CONFLICT
```

NO:

```
stock = -1
```

La operación conflictiva debe conservarse.

Debe mostrar:

- usuario
- producto
- cantidad
- fecha
- motivo
- causa del conflicto

Admin podrá resolver explícitamente.

Nunca resolver silenciosamente modificando historia.

24. INDEXEDDB

Puede utilizarse en frontend como resiliencia adicional.

Por ejemplo:

- operación que todavía no llegó al backend
- estado UI
- información temporal útil

Pero IndexedDB NO reemplaza la durable outbox del backend.

La garantía principal una vez recibido el request debe estar en servidor.

25. AUTENTICACIÓN

Implementar autenticación segura.

Preferir sesiones mediante cookies.

Cookies:

- HttpOnly
- SameSite apropiado
- Secure cuando el contexto lo permita

No almacenar tokens sensibles de larga duración en localStorage.

Passwords:

- Argon2id
- nunca texto plano

Nunca devolver:

`passwordHash`

al frontend.

26. USUARIOS

Modelo mínimo:

```
{
  _id,
  name,
  email,
  passwordHash,
  role,
  active,
  createdAt,
  updatedAt
}
```

Email UNIQUE.

Usuario desactivado:

- no inicia sesión
- historial permanece

27. PERMISOS

Definir permisos internos.

Ejemplo:

```
product:read
product:create
product:update

stock:out
stock:in
stock:return
stock:adjust

movement:read-own
movement:read-all

user:manage
```



```
audit:read  
system:read
```

Separar:

```
ROLE
```

de:

```
PERMISSION
```

para facilitar crecimiento futuro.

28. MIDDLEWARES

Como mínimo:

```
middlewares/  
├─ auth.middleware.js  
├─ authorization.middleware.js  
├─ requestId.middleware.js  
├─ validation.middleware.js  
├─ rateLimit.middleware.js  
├─ notFound.middleware.js  
└─ error.middleware.js
```

Middlewares para concerns transversales.

No colocar negocio allí.

29. SEGURIDAD

Implementar:

- Helmet
- CORS restringido
- rate limiting
- Zod

- validación backend
- sanitización donde corresponda
- protección contra NoSQL injection
- límites de request
- manejo seguro de sesiones
- secretos mediante env
- no stack traces al usuario en production
- manejo correcto de cookies
- expiración de sesiones
- invalidación adecuada
- autorización backend
- validación de ObjectId
- principio de mínimo privilegio

No confiar en datos provenientes del cliente para:

- userId
- rol
- timestamps oficiales
- stock resultante
- stock anterior
- permisos

El backend determina esos valores.

30. REQUEST ID

Cada request debe tener un ID.

Ejemplo:

```
req_...
```

Debe propagarse a:

- logs
- errores
- auditoría cuando corresponda

Respuesta de error:

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
```

```
{
  "message": "No hay stock suficiente.",
  "details": {
    "available": 3,
    "requested": 5
  },
  "requestId": "req_..."
}
```

Nunca mostrar solamente:

```
Error 500
```

31. ERRORES

Crear jerarquía consistente de errores.

Ejemplos:

```
VALIDATION_ERROR
UNAUTHORIZED
FORBIDDEN
PRODUCT_NOT_FOUND
PRODUCT_INACTIVE
INSUFFICIENT_STOCK
DUPLICATE_BARCODE
DUPLICATE_INTERNAL_CODE
DUPLICATE_OPERATION
SYNC_FAILED
SYNC_CONFLICT
DATABASE_UNAVAILABLE
```

Los errores internos deben contener información diagnóstica.

Los errores públicos deben ser comprensibles y seguros.

32. LOGGING

Logging estructurado.

Preferentemente JSON.

Niveles:

```
debug
info
warn
error
```

Registrar eventos como:

```
auth.login.success
auth.login.failed

product.created
product.updated
product.deactivated

inventory.out
inventory.in
inventory.return
inventory.adjustment

sync.started
sync.success
sync.failed
sync.conflict

mongodb.unavailable
mongodb.recovered
```

Incluir cuando corresponda:

- requestId
- operationId
- userId
- productId
- endpoint
- método
- timestamp

Nunca loguear:

- password
- passwordHash

- cookies
- session secrets
- tokens
- Mongo URI completa
- secrets

Implementar rotación de logs.

Persistir logs mediante volumen Docker.

33. AUDITORÍA

Crear `auditLogs` .

Debe registrar:

```
{
  userId,
  action,
  entity,
  entityId,
  changes,
  reason,
  requestId,
  operationId,
  createdAt
}
```

Cuando corresponda.

Ejemplo:

```
{
  action: "PRODUCT_UPDATED",
  changes: {
    minimumStock: {
      from: 5,
      to: 10
    }
  }
}
```

AuditLog:

- no editable
- no borrrable desde aplicación

34. CONTEO FÍSICO

Implementar inventario físico.

Ejemplo:

```
Sistema: 47  
Físico: 45  
Diferencia: -2
```

Si admin confirma:

```
ADJUSTMENT_OUT 2
```

Motivo obligatorio.

Nunca:

```
product.stock = 45
```

sin movimiento.

35. LECTOR DE CÓDIGO DE BARRAS

El lector funciona como USB HID Keyboard.

NO implementar drivers especiales.

Crear componente reusable para scanner.

Debe:

- mantener foco
- aceptar caracteres rápidamente

- detectar Enter
- prevenir doble procesamiento accidental
- permitir escaneo consecutivo
- dar feedback inmediato

No asumir que todos los códigos existen en Google o en servicios externos.

Nuestra base de productos es la fuente de identificación.

36. FLUJO DEL TÉCNICO

Debe ser extremadamente rápido.

```
Login
↓
Registrar salida
↓
Escanear
↓
Producto encontrado
↓
Cantidad
↓
Motivo
↓
Destino opcional
↓
Confirmar
↓
Feedback
↓
Scanner listo nuevamente
```

Evitar navegación innecesaria.

37. UX/UI

UX es importante.

La aplicación será utilizada repetidamente durante el trabajo.

Priorizar:

- velocidad
- claridad
- legibilidad
- baja carga cognitiva
- prevención de errores
- feedback inmediato
- accesibilidad básica
- navegación consistente

La estética debe ser:

- moderna
- profesional
- sobria
- limpia
- compacta
- consistente

Evitar:

- apariencia genérica de template
- exceso de colores
- gradientes decorativos
- sombras exageradas
- animaciones innecesarias
- espacios enormes
- interfaces infantiles
- dashboards llenos de métricas irrelevantes

Utilizar Tailwind CSS.

Crear un sistema visual consistente.

38. COMPONENTES UI

Como mínimo considerar:

```
components/ui/  
├─ Button.jsx  
├─ Input.jsx  
├─ Select.jsx  
├─ Modal.jsx  
└─ Dialog.jsx
```



```
├─ Table.jsx
├─ Badge.jsx
├─ Alert.jsx
├─ Card.jsx
├─ Spinner.jsx
├─ EmptyState.jsx
├─ Pagination.jsx
└─ ConfirmDialog.jsx
```

Reutilizar.

No copiar enormes cadenas Tailwind por toda la aplicación cuando puede existir un componente.

39. ACCIONES DESTRUCTIVAS / IMPORTANTES

Aunque no haya DELETE, algunas operaciones requieren cuidado.

Ejemplos:

- desactivar producto
- desactivar usuario
- ajuste de stock
- resolver conflicto

Usar confirmaciones claras.

No usar confirmaciones para cada acción trivial.

40. ESTADO DE CONECTIVIDAD

Mostrar discretamente:

```
Online
Todo sincronizado
```

o:

```
MongoDB sin conexión
3 operaciones pendientes
```

También:

2 conflictos

No confiar exclusivamente en:

`navigator.onLine`

Consultar `/api/health`.

41. HEALTH CHECK

Crear:

`GET /api/health`

Respuesta aproximada:

```
{
  "status": "degraded",
  "api": "ok",
  "mongodb": "offline",
  "outbox": {
    "pending": 3,
    "syncing": 0,
    "failed": 0,
    "conflicts": 0
  },
  "version": "1.0.0"
}
```

Estados:

healthy
degraded
unhealthy

42. DASHBOARD ADMIN

Mostrar información operacional.

Como mínimo:

- salidas hoy
- entradas hoy
- stock bajo
- sin stock
- pendientes
- conflictos
- últimos movimientos
- estado sistema

Accesos rápidos:

- entrada
- salida
- productos
- movimientos
- conteo físico
- usuarios
- auditoría

No agregar gráficos decorativos si no aportan información.

43. HISTORIAL

Filtros:

- fecha desde
- fecha hasta
- usuario
- producto
- tipo
- motivo
- cliente
- operationId
- estado

Mostrar:

- fecha
- hora

- usuario
- producto
- cantidad
- tipo
- stock anterior
- stock resultante
- motivo
- destino
- estado

No permitir:

- editar
 - borrar
-

44. PAGINACIÓN

Implementar paginación server-side para:

- productos
- movimientos
- auditoría
- usuarios

No cargar colecciones completas innecesariamente.

45. ÍNDICES MONGODB

Diseñar conscientemente.

Revisar índices para:

- operationId
- barcodes
- internalCode
- email
- createdAt
- productId
- userId
- type
- active

No crear índices arbitrariamente.

46. CONTRATOS COMPARTIDOS

Utilizar:

```
/shared/schemas
```

para esquemas Zod compatibles cuando tenga sentido.

Ejemplo:

```
export const stockOutSchema = z.object({
  operationId: z.string().uuid(),
  productId: z.string().min(1),
  quantity: z.number().int().positive(),
  reason: z.string().trim().min(1),
  destination: z.string().trim().optional()
});
```

Frontend valida para UX.

Backend vuelve a validar SIEMPRE.

47. RESPUESTAS API

Éxito:

```
{
  "data": {}
}
```

Error:

```
{
  "error": {
    "code": "...",
    "message": "...",
    "details": {},
    "requestId": "..."
  }
}
```

```
}  
}
```

Utilizar correctamente status HTTP.

48. GRACEFUL SHUTDOWN

Manejar:

```
SIGTERM  
SIGINT
```

Secuencia:

```
recibir señal  
↓  
dejar de aceptar nuevos requests  
↓  
terminar requests activos  
↓  
detener sync worker  
↓  
liberar locks  
↓  
cerrar SQLite  
↓  
cerrar MongoDB  
↓  
cerrar servidor  
↓  
exit
```

Evitar cortar operaciones a mitad.

49. DOCKER

Crear:

- Dockerfile frontend
- Dockerfile backend

- docker-compose.yml
- `.dockerignore`

Servicios mínimos:

```
frontend  
backend
```

MongoDB Atlas NO corre en Docker.

SQLite debe utilizar volumen persistente.

Logs deben utilizar volumen persistente.

Utilizar:

```
restart: unless-stopped
```

La aplicación debe quedar accesible desde la LAN.

No depender de localhost internamente de manera incorrecta.

50. BACKUPS

MongoDB Atlas remoto no elimina la necesidad de backup.

Crear scripts documentados utilizando `mongodump`.

Estrategia inicial:

```
7 diarios  
4 semanales
```

Guardar en ubicación configurable.

Documentar cómo automatizarlo en la PC host.

No hardcodear credenciales.

51. TESTING

No escribir tests decorativos.

Testear comportamiento real.

Backend

Vitest + Supertest.

Frontend

Vitest + React Testing Library.

E2E

Playwright.

52. TESTS OBLIGATORIOS

Como mínimo:

- ✓ login correcto
- ✓ login incorrecto
- ✓ usuario desactivado

- ✓ permisos TECHNICIAN
- ✓ permisos ADMIN

- ✓ técnico no puede registrar entrada
- ✓ técnico no puede administrar usuarios

- ✓ crear producto
- ✓ barcode duplicado
- ✓ internalCode duplicado
- ✓ producto inactivo

- ✓ entrada
- ✓ salida
- ✓ devolución
- ✓ ajuste

- ✓ cantidad 0 rechazada
- ✓ cantidad negativa rechazada
- ✓ producto inexistente
- ✓ stock insuficiente

- ✓ stock nunca negativo

- ✓ movement append-only
- ✓ no existe DELETE movement
- ✓ corrección mediante movimiento compensatorio

- ✓ operationId única
- ✓ mismo request enviado dos veces
- ✓ mismo request enviado cinco veces
- ✓ doble click

- ✓ concurrencia de dos técnicos
- ✓ atomicidad de stock

- ✓ MongoDB offline
- ✓ outbox persiste
- ✓ reinicio backend
- ✓ outbox continúa existiendo
- ✓ MongoDB vuelve
- ✓ sync funciona
- ✓ no duplica movimiento

- ✓ operación Mongo aplicada + respuesta perdida
- ✓ retry no vuelve a descontar

- ✓ conflicto offline
- ✓ conflicto queda visible
- ✓ conflicto no altera stock incorrectamente

- ✓ producto desactivado conserva historial
- ✓ usuario desactivado conserva historial

- ✓ health endpoint
- ✓ RBAC
- ✓ error responses
- ✓ requestId

53. TESTS DE CONCURRENCIA

Crear tests específicos para race conditions.

Ejemplo:

```
stock inicial = 1  
  
request A → OUT 1  
request B → OUT 1
```

Resultado obligatorio:

```
uno exitoso  
uno rechazado  
stock final = 0
```

Nunca:

```
stock = -1
```

54. TESTS OFFLINE

Simular realmente indisponibilidad MongoDB.

No mockear absolutamente todo de forma que el test pierda valor.

Probar:

```
Mongo offline  
↓  
OUT  
↓  
SQLite PENDING  
↓  
reinicio  
↓  
sigue PENDING  
↓  
Mongo online  
↓  
SYNCED
```

55. CI

GitHub Actions.

Ejecutar:

```
npm run lint
npm run test
npm run test:integration
npm run build
npm run test:e2e
```

Si algo falla:

```
pipeline FAILED
```

No considerar versión válida.

56. COVERAGE

Configurar cobertura.

No perseguir 100% artificial.

Exigir cobertura especialmente alta en:

- inventory services
- idempotency
- authentication
- authorization
- synchronization
- conflict handling
- outbox

Los tests deben cubrir riesgos, no simplemente líneas.

57. SEED

Crear seed para development/test.

Incluir:

- admin
- technician
- categorías
- ubicaciones
- algunos productos

No utilizar credenciales inseguras como defaults de producción.

No ejecutar seed automáticamente en production.

58. VARIABLES DE ENTORNO

Crear `.env.example`.

Como mínimo:

```
NODE_ENV
PORT
FRONTEND_ORIGIN
MONGODB_URI
SESSION_SECRET
SQLITE_PATH
LOG_LEVEL
LOG_PATH
BACKUP_PATH
```

Agregar las necesarias.

Validarlas al arrancar.

Si falta una variable crítica:

```
fail fast
```

con error claro.

Nunca commitear `.env`.

59. CALIDAD DE CÓDIGO

Código:

- limpio
- legible
- modular
- consistente
- mantenible

Preferir:

- funciones pequeñas
- nombres explícitos
- responsabilidades claras
- early returns cuando mejoren legibilidad
- constantes para valores importantes
- módulos cohesionados

Evitar:

- archivos gigantes
- funciones gigantes
- magic numbers
- duplicación
- abstracciones prematuras
- dependencias innecesarias
- código muerto
- comentarios obvios

No dejar:

```
TODO  
FIXME  
placeholder  
implement later
```

en funcionalidades consideradas terminadas.

60. CONTROL DE CAMBIOS

Mantener cambios pequeños y coherentes.

No reescribir archivos completos innecesariamente.

Antes de refactorizaciones importantes:

```
tests actuales deben pasar
```

Después:

```
tests deben seguir pasando
```

No modificar configuraciones externas al proyecto.

61. ACCESIBILIDAD

Aplicar accesibilidad básica correctamente:

- labels
- focus visible
- navegación por teclado
- contraste adecuado
- botones semánticos
- formularios accesibles
- errores asociados al campo
- modales con manejo de focus

El scanner no debe destruir la navegación normal por teclado.

62. PERFORMANCE

No realizar optimizaciones prematuras.

Sí evitar:

- queries N+1 innecesarias
 - consultas sin límites
 - colecciones completas
 - renders React innecesarios obvios
 - polling agresivo
 - sincronización en loops sin backoff
-

63. README

Mantener README actualizado.

Debe explicar:

- arquitectura
- requisitos
- instalación
- variables env
- MongoDB Atlas
- Docker
- Docker Compose
- desarrollo
- producción local
- tests
- backups
- restauración
- logs
- health check
- troubleshooting
- funcionamiento offline
- recuperación después de caída

Una persona distinta al desarrollador debe poder levantar el sistema siguiendo README.

64. DESARROLLO POR FASES

Trabajar en este orden.

FASE 1 — FOUNDATION

Crear:

- monorepo
- frontend
- backend
- Docker
- configuración
- MongoDB
- SQLite
- logging
- errores
- requestId

- health
- graceful shutdown
- lint
- testing base

Ejecutar:

```
lint
tests
build
```

Corregir antes de continuar.

FASE 2 — AUTH

Implementar:

- usuarios
- passwords
- sesiones
- roles
- permisos
- middlewares

Tests.

FASE 3 — PRODUCTOS

Implementar:

- productos
- categorías
- ubicaciones
- internalCode
- barcodes
- active
- desactivar/reactivar

Sin DELETE.

Tests.

FASE 4 — INVENTARIO

Implementar:

- IN
- OUT
- RETURN
- ADJUSTMENT_IN
- ADJUSTMENT_OUT
- atomicidad
- transacciones
- idempotencia
- auditoría

Tests incluyendo concurrencia.

FASE 5 — OFFLINE

Implementar:

- SQLite durable outbox
- sync manager
- retry
- backoff
- recovery
- conflict
- IndexedDB complementario

Tests de caída/reinicio/reconexión.

FASE 6 — UI

Implementar:

- login
- layout
- dashboard
- scanner
- salida
- entrada
- productos
- movimientos
- usuarios
- auditoría
- conteo físico
- sync status
- conflicts

Aplicar UX definida.

FASE 7 — E2E

Implementar escenarios completos Playwright.

FASE 8 — HARDENING

Revisar:

- seguridad
- performance
- accesibilidad
- logs
- backups
- recuperación
- Docker
- documentación
- UX
- edge cases

65. PROCEDIMIENTO OBLIGATORIO POR FASE

Al terminar cada fase:

1. ejecutar lint
2. ejecutar tests unitarios
3. ejecutar integración cuando corresponda
4. ejecutar build
5. ejecutar E2E cuando corresponda
6. revisar errores
7. corregirlos
8. volver a ejecutar
9. comprobar regresiones
10. continuar automáticamente

No pidas autorización para continuar si todo está correcto.

66. DEFINITION OF DONE

El proyecto NO está terminado hasta verificar realmente:

- ✓ docker compose levanta
- ✓ frontend funciona
- ✓ backend funciona
- ✓ aplicación accesible desde LAN
- ✓ MongoDB conecta
- ✓ SQLite persiste tras reiniciar contenedor
- ✓ login funciona
- ✓ RBAC funciona
- ✓ productos funcionan
- ✓ desactivar/reactivar funciona
- ✓ no existen DELETE prohibidos
- ✓ entrada funciona
- ✓ salida funciona
- ✓ devolución funciona
- ✓ ajustes funcionan
- ✓ scanner HID funciona mediante entrada de teclado
- ✓ movimientos son append-only
- ✓ idempotencia funciona
- ✓ doble request no duplica
- ✓ concurrencia funciona
- ✓ stock nunca es negativo
- ✓ offline funciona
- ✓ outbox persiste
- ✓ reinicio no pierde pendientes

- ✓ resincronización funciona
- ✓ operación no se duplica
- ✓ conflictos funcionan
- ✓ auditoría funciona
- ✓ health funciona
- ✓ logs funcionan
- ✓ backups están implementados/documentados
- ✓ lint pasa
- ✓ tests pasan
- ✓ integration tests pasan
- ✓ E2E pasa
- ✓ build pasa
- ✓ README está completo

67. PRINCIPIO DE FALLOS

No existe software que nunca falle.

Por lo tanto, diseña el sistema para que los fallos sean:

```
detectables
↓
registrados
↓
comprensibles
↓
recuperables
↓
trazables
```

Un fallo de:

- Internet
- MongoDB
- navegador
- backend
- Docker
- PC

no debe provocar silenciosamente:

- pérdida de movimientos
- movimientos duplicados
- stock negativo
- historial alterado

68. INVARIANTES DEL SISTEMA

Estas reglas tienen máxima prioridad:

1. Stock nunca puede ser negativo.
2. Un operationId nunca puede afectar stock más de una vez.
3. Un movimiento confirmado nunca se modifica.
4. Un movimiento confirmado nunca se elimina.
5. Las correcciones generan nuevos movimientos.
6. Usuarios históricos nunca desaparecen.
7. Productos históricos nunca desaparecen.
8. El backend determina identidad y permisos.
9. El frontend nunca determina el stock definitivo.
10. MongoDB es la fuente de verdad del stock confirmado.
11. SQLite es durable outbox, no segunda fuente de verdad.
12. Una operación pendiente nunca debe presentarse como sincronizada.
13. Un conflicto nunca se resuelve silenciosamente.

- 14. Toda operación crítica debe ser auditable.
- 15. Toda pérdida de conectividad debe ser visible.
- 16. Toda operación offline aceptada debe sobrevivir reinicios.
- 17. Los retries deben ser idempotentes.
- 18. Los errores deben ser trazables mediante requestId/operationId.

Si una implementación viola alguna de estas invariantes, está incorrecta aunque los tests existentes pasen.

69. CRITERIO PARA DECISIONES NO ESPECIFICADAS

Si surge una decisión no cubierta aquí:

Priorizar en este orden:

```
integridad de datos
seguridad
trazabilidad
idempotencia
recuperabilidad
simplicidad
mantenibilidad
UX
performance
```

No introducir infraestructura adicional sin necesidad demostrable.

No sobreingenierizar.

70. PRIMERA ACCIÓN

Antes de escribir grandes cantidades de código:

1. inspecciona el repositorio actual
2. determina si está vacío o contiene código existente
3. conserva cualquier código válido que sea compatible con esta especificación
4. define brevemente el plan de implementación

5. comienza inmediatamente con FASE 1

No me pidas confirmación salvo que exista un bloqueo real que no pueda resolverse razonablemente.

Continúa autónomamente fase por fase.

La meta final es entregar un sistema de stock local, robusto, auditable, seguro, offline-capable y listo para ser utilizado realmente por la empresa.